ANHANGUERA

MICHELA JULIANA OLIVEIRA

MATRICULA- 2025369608

RELATÓRIO DE AULA PRÁTICA

Unidade 3 – Classes e Interfaces

SERTÃOZINHO -SP

2026

MICHELA JULIANA OLIVEIRA

MATRICULA- 2025369608

RELATÓRIO DE AULA PRÁTICA

Unidade 3 – Classes e Interfaces

Trabalho de Aula Prática apresentado ao curso de Bacharelado em Engenharia de Software da Faculdade Anhanguera, como requisito parcial para obtenção de nota na disciplina de Linguagem Orientada A Objetos (Unidade 3 – Estruturas Do Desenvolvimento Orientado a Objetos).

SERTÃOZINHO -SP

2026

SUMÁRIO

1. INTRODUÇÃO ____ *4*

2. OBJETIVO ____ *5*

3. METODOLOGIA E DESENVOLVIMENTO ____ *6*

4. RESULTADO ____ *8*

4.1 Arquivo OperacaoMatematica.java ____ *9*

4.2 Arquivo Soma.java ____ *9*

4.3 Arquivo Divisao.java ____ *9*

4.4 Arquivo DivisaoPorZeroException.java ____ *10*

4.5 Arquivo Principal.java ____ *10*

5. CONCLUSÃO ____ *12*

6. REFERÊNCIAS ____ *13*

1. INTRODUÇÃO

O paradigma de Orientação a Objetos (POO) fornece uma estrutura robusta para a criação de sistemas de software modulares, flexíveis e de fácil manutenção (DEITEL; DEITEL, 2016). Entre seus pilares fundamentais, destaca-se a abstração, que permite a representação de modelos conceituais genéricos através de classes abstratas, estabelecendo contratos que devem ser obrigatoriamente cumpridos por suas subclasses concretas via herança e polimorfismo (SIERRA; BATES, 2007).

Complementarmente à estruturação do código, o desenvolvimento de sistemas profissionais exige um mecanismo eficiente para o manuseio de erros em tempo de execução (ORACLE, 2026). Em Java, o tratamento de exceções permite desviar o fluxo do programa diante de situações anômalas, garantindo a integridade da aplicação e fornecendo mensagens amigáveis ao usuário final (DEITEL; DEITEL, 2016).

Neste contexto, o presente relatório descreve o desenvolvimento prático de uma aplicação em linguagem Java focada no cálculo de operações matemáticas. A solução integra a aplicação de classe abstrata, implementação de subclasses concretas (Soma e Divisao), criação de uma exceção personalizada (DivisaoPorZeroException) e a captura segura de falhas utilizando blocos try-catch.

2. OBJETIVO

O objetivo deste trabalho é aplicar de forma prática e integrada os conceitos de orientação a objetos e tratamento de exceções em Java, desenvolvendo um sistema de operações matemáticas com herança, classe abstrata, exceção personalizada e estrutura try-catch.

3. METODOLOGIA E DESENVOLVIMENTO

Para a realização da atividade prática, foi utilizada a linguagem de programação Java e a ferramenta Eclipse IDE for Java Developers. O desenvolvimento teve como finalidade aplicar, de maneira prática e integrada, os conceitos de programação orientada a objetos, especialmente classe abstrata, herança, polimorfismo e tratamento de exceções.

Inicialmente, foi criado um projeto Java denominado OperacoesMatematicas, juntamente com o pacote br.edu.operacoes, responsável por organizar as classes desenvolvidas. Em seguida, foi criada a classe abstrata OperacaoMatematica, utilizada como estrutura base para as operações matemáticas. Nessa classe foi definido o método abstrato calcular(double a, double b), estabelecendo um padrão que deveria ser implementado pelas classes derivadas.

Posteriormente, foram desenvolvidas as classes Soma e Divisao, ambas utilizando herança a partir da classe abstrata OperacaoMatematica. A classe Soma foi implementada para realizar a adição de dois valores recebidos como parâmetros. Já a classe Divisao foi desenvolvida para realizar a divisão, sendo incluída uma verificação do segundo valor recebido. Quando o divisor apresenta valor igual a zero, o programa interrompe a operação e lança uma exceção específica para essa situação.

Para realizar o tratamento da divisão por zero, foi criada a classe personalizada DivisaoPorZeroException, estendendo a classe Exception da linguagem Java. Essa exceção recebeu uma mensagem específica, informando ao usuário que não é possível realizar uma divisão por zero. Dessa forma, foi possível representar de maneira específica a situação de erro prevista na atividade.

Na sequência, foi criada a classe Principal, contendo o método main, responsável por executar e testar as operações implementadas. Foram instanciados objetos das classes Soma e Divisao utilizando referências do tipo OperacaoMatematica, permitindo demonstrar o uso de herança e polimorfismo. Foram realizados testes com valores definidos para verificar o funcionamento da soma e da divisão.

Também foi realizado um teste específico com uma tentativa de divisão por zero. Para isso, a chamada do método de divisão foi envolvida em uma estrutura try-catch, permitindo que a exceção DivisaoPorZeroException fosse capturada e tratada sem interromper de forma abrupta a execução do programa. A mensagem de erro foi apresentada no console do Eclipse, possibilitando verificar se o tratamento da exceção estava funcionando corretamente.

Após a implementação das classes e dos testes, o programa foi executado no Eclipse para verificar os resultados obtidos. A execução confirmou o funcionamento das operações

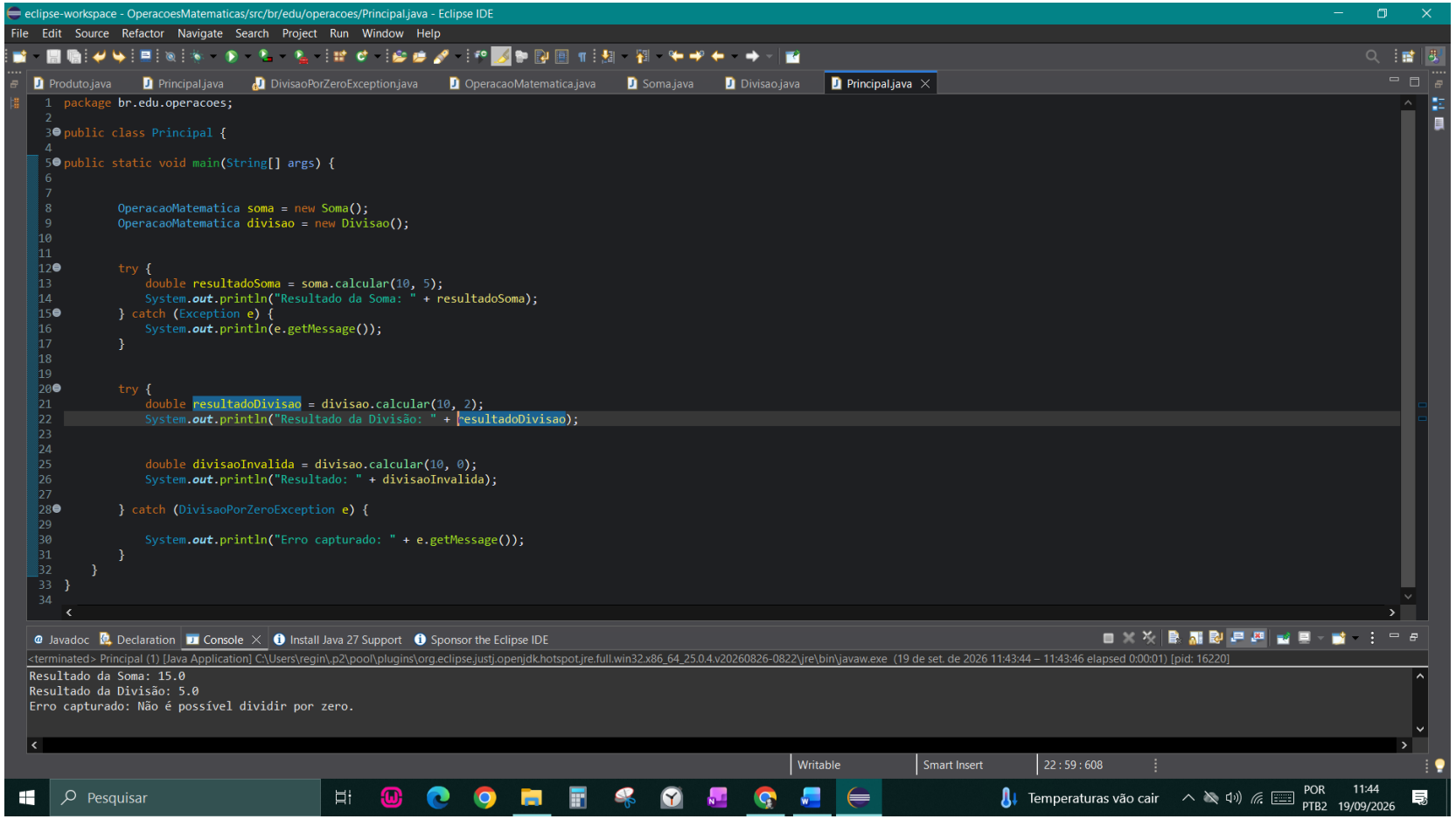
matemáticas e do tratamento da exceção personalizada, sendo registrada por meio de um print do console para servir como evidência do desenvolvimento e dos resultados alcançados.

Dessa maneira, a metodologia empregada possibilitou aplicar na prática os principais conceitos propostos na atividade, relacionando a estrutura de classes e herança ao tratamento adequado de situações excepcionais durante a execução de um programa Java.

4. RESULTADO

Na Imagem 1, está apresentada a evidência de saída obtida no console do Eclipse IDE durante a execução da classe Principal. A execução demonstra o cálculo correto das operações de soma e divisão, bem como a captura e o tratamento da exceção personalizada**.**

Imagem 1- Execução do programa no Eclipse



4.1 Arquivo OperacaoMatematica.java

A seguir está o código utilizado:

```
package br.edu.operacoes;

public abstract class OperacaoMatematica {

    public abstract double calcular(double a, double b) throws DivisaoPorZeroException;
}
```

4.2 Arquivo Soma.java

A seguir está o código utilizado:

```
package br.edu.operacoes;

public class Soma extends OperacaoMatematica {

    @Override
    public double calcular(double a, double b) throws DivisaoPorZeroException {
        return a + b;
    }
}
```

4.3 Arquivo Divisao.java

A seguir está o código utilizado:

```
package br.edu.operacoes;
```

```
public class Divisao extends OperacaoMatematica {

    @Override
    public double calcular(double a, double b) throws DivisaoPorZeroException {
        if (b == 0) {
            throw new DivisaoPorZeroException();
        }
        return a / b;
    }
}
```

4.4 Arquivo DivisaoPorZeroException.java

A seguir está o código utilizado:

```
        package br.edu.operacoes;

        public class DivisaoPorZeroException extends Exception {

            public DivisaoPorZeroException() {
                super("Não é possível dividir por zero.");
            }
        }
```

4.5 Arquivo Principal.java

A seguir está o código utilizado:

```
package br.edu.operacoes;

public class Principal {
```

```
public static void main(String[] args) {

        OperacaoMatematica soma = new Soma();
        OperacaoMatematica divisao = new Divisao();

        try {
            double resultadoSoma = soma.calcular(10, 5);
            System.out.println("Resultado da Soma: " + resultadoSoma);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        try {
            double resultadoDivisao = divisao.calcular(10, 2);
            System.out.println("Resultado da Divisão: " + resultadoDivisao);

            double divisaoInvalida = divisao.calcular(10, 0);
            System.out.println("Resultado: " + divisaoInvalida);

        } catch (DivisaoPorZeroException e) {

            System.out.println("Erro capturado: " + e.getMessage());
        }
    }
}
```

5. CONCLUSÃO

A realização da atividade permitiu aplicar, na prática, os conceitos de programação orientada a objetos em Java, especialmente classes abstratas, herança e polimorfismo. O desenvolvimento das classes Soma e Divisao possibilitou realizar operações matemáticas, enquanto a criação da exceção personalizada DivisaoPorZeroException permitiu tratar adequadamente a tentativa de divisão por zero. Dessa forma, a atividade contribuiu para a compreensão da utilização de exceções e do mecanismo try-catch na construção de aplicações mais seguras e organizadas.

6. REFERÊNCIAS

DEITEL, Paul; DEITEL, Harvey. **Java: Como Programar**. 10. ed. São Paulo: Pearson Education do Brasil, 2016.

ORACLE. **Java Platform, Standard Edition Documentation**. Oracle Corporation, 2026. Disponível em: https://docs.oracle.com/en/java/. Acesso em: 19 set. 2026.

SIERRA, Kathy; BATES, Bert. **Use a Cabeça! Java**. 2. ed. Rio de Janeiro: Alta Books, 2007.